SC-100: Designing Security Solutions for Applications and Data

Design an application security program for the release train



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com





Checking in with Globomantics



Connected Equipment ships every two weeks

Its lead says a gate that adds a day breaks the train

Legacy apps ship twice a year, and headcount is flat

One gate may block a release. Where does it go?

Assess each estate with the product that sees it

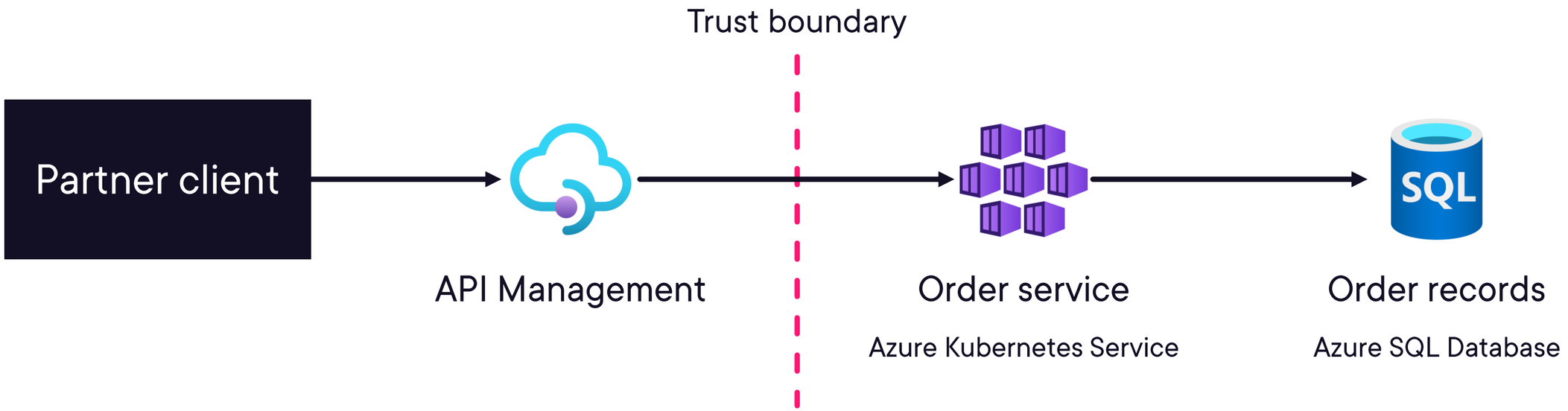
Connected Equipment API
Assess with: Defender CSPM attack path analysis
First: fix the exploitable path to orders

Legacy internal apps
Assess with: Defender for Cloud recommendations
First: contain exposure while the owner plans a fix

SaaS applications
Assess with: Defender for Cloud Apps
First: review OAuth grants and app governance

Model threats to a business-critical application

Follow the order cancellation across a trust boundary



Each flow across it needs authentication, authorization, and validation

Turn STRIDE findings into testable requirements

Spoofing and elevation of privilege
Ask: can one partner act on another's orders?
Require: validate the token and authorize the order

Tampering and information disclosure
Ask: can input or responses expose other orders?
Require: validate input and limit what returns

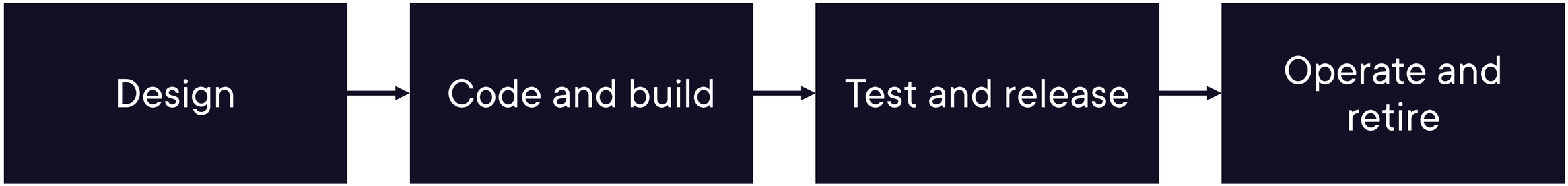
Repudiation and denial of service
Ask: can a cancellation be disputed or abused?
Require: record actor, order, action, and outcome

Demo: Threat model the Connected Equipment API

Read the trust boundary, generate the STRIDE threats, and pick this sprint's requirement

Design a full lifecycle application security strategy

Carry the requirement through the lifecycle



The same requirement shows up in the backlog, the test, and the evidence

An emergency release still needs evidence

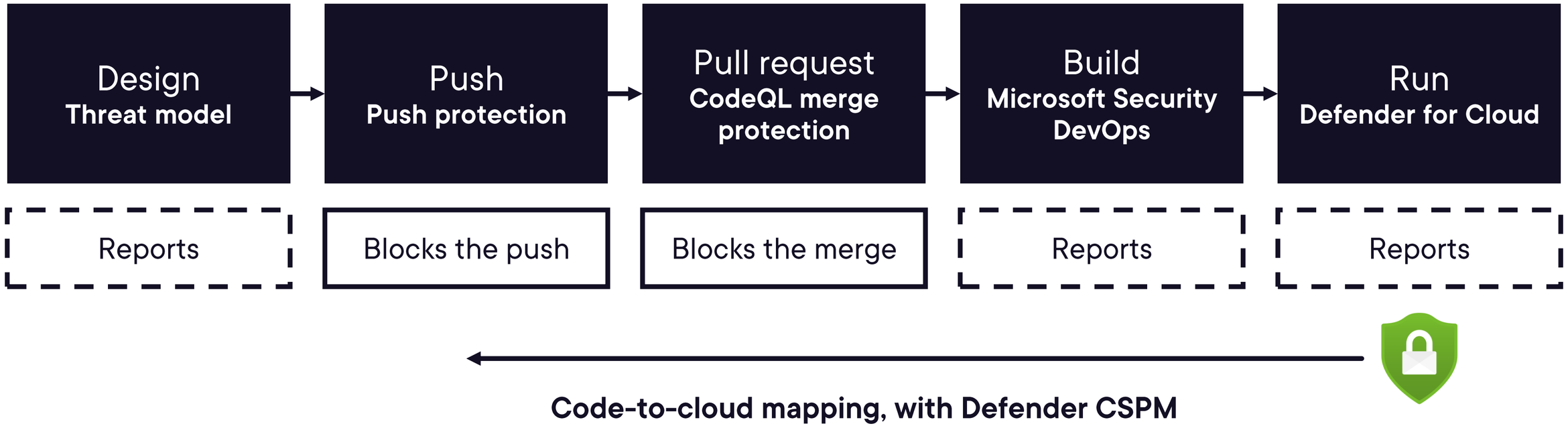
Routine release	vs.	Emergency release
Automated tests		Expedited approval
Traceable artifact		Tests still run
Normal approval route		Owned, time-bounded exception

Set standards for the secure development process

Make the development standard enforceable

Standard	Enforced by	Evidence it ran
No supported secrets in a push	Secret scanning push protection	A blocked push or a reviewed bypass
No critical code findings in main	CodeQL merge protection in a ruleset	A blocked merge until results pass
Secrets come from a vault at run time	Azure Key Vault and workload identity federation	No stored credential in the pipeline

Place each gate on the release train



Route each finding to an owner

Source: repository and pipeline run

Runtime: the deployed image or resource

Owner: a fix, or an exception that expires



Practice question and key points

Theme: Secure development process standards

Question

Globomantics uses Azure Pipelines to deploy Bicep templates for the Materials business unit. The pipeline runs each time a developer merges a pull request to the main branch.

You need to recommend a solution that meets the following requirements:

- **Scan each Bicep template for misconfigurations during each pipeline run.**
- **Fail the pipeline run when the scan finds a high-severity misconfiguration.**



Theme: Secure development process standards

Question

You need to recommend a solution for the Materials pipeline. What should you include in the recommendation?

A. secret scanning with push protection

B. the Microsoft Security DevOps extension

C. code scanning by using CodeQL

D. agentless code scanning in Microsoft Defender for Cloud



Theme: Secure development process standards

Answer: B

You need to recommend a solution for the Materials pipeline. What should you include in the recommendation?

A. secret scanning with push protection

B. the Microsoft Security DevOps extension

C. code scanning by using CodeQL

D. agentless code scanning in Microsoft Defender for Cloud

Checking out with Globomantics



Tier the portfolio and rank fixes by attack path

Model threats at design, before code exists

Carry each requirement through every stage

Block at the pull request, and let the rest report